

Systems Verification

Exercise Sheet 05 - Solutions

Exercise 1: Understanding Linear Temporal Logic

Are the following equivalences of LTL valid? If yes, prove both directions of the implication, if not, give a counter-example.

1. $\Diamond p \wedge q \longleftrightarrow \Diamond(p \wedge q)$,

Solution: No. Consider for instance a state sequence $x = s_0 s_1 s_2 \dots$ where p is false in s_1 and true afterwards and q is true only in s_1 . Then $x \not\models \Diamond(p \wedge q)$ because p and q are never true at the same time. However, x satisfies $\Diamond p \wedge q$ because $x \models \Diamond p$ (p being true in at least one state) and $x \models q$ if and only if q is true in s_0 , which is the case.

2. $\Box p \wedge \Box q \longleftrightarrow \Box(p \wedge q)$,

Solution: Yes. Indeed, $\Box p \wedge \Box q$ is true if and only if p is always true and q is always true. This is equivalent to $p \wedge q$ being always true, which by definition is represented by the LTL-formula $\Box(p \wedge q)$.

3. $\Diamond(q \mathcal{U} p) \longleftrightarrow \Diamond q$,

Solution: No. This because $\Diamond(q \mathcal{U} p)$ is satisfied for example by a state sequence x where p is always true and q is always false, but x does not satisfy $\Diamond q$.

Exercise 2: Never claim and the corresponding LTL formula

- Let q be an atomic proposition for some model and consider the LTL formula $\Diamond\Box q$. Write your own corresponding *never claim* that assures satisfiability of the formula in the model.
- Use the `-f` spin command and compare this to the solution you gave to the same LTL formula in the previous step.

Solution:

Since we want our model to satisfy $\Diamond\Box q$ (in English: eventually q is always true), we first have to *negate* it to get the formula that the never claim will check. Then, we generate the never claim corresponding to $\neg\Diamond\Box q$ using the command line. From:

```
spin -f "!<>[] q"
```

we get:

```

never {    /* !<>[] q */
T0_init:
    do
        :: (! ((q))) -> goto accept_S9
        :: (1) -> goto T0_init
    od;
accept_S9:
    do
        :: (1) -> goto T0_init

```

```

    od;
}

```

Notice that $\neg\Diamond\Box q \equiv \Box\Diamond\neg q$, i.e. the never claim checks that q is false infinite many times.

Exercise 3: Formalizing properties with LTL formulas

Let q, p be an atomic proposition for some model. Construct LTL formulas that encode the following properties of our system (where "time" is measured as execution steps):

- p must be true at the 3rd execution step.

Solution: $X(X(p))$

- p must be true at ONLY the 3rd execution step.

Solution: $\neg p \wedge X(\neg p \wedge X(p \wedge X(\Box\neg p)))$

- q needs to hold for every execution step and p has to be true infinitely often during the execution of our system.

Solution: $\Box q \wedge (\Box\Diamond p)$

- The first time in our execution that q is true, we know that from the next execution step onwards, p will never be false.

Solution: $\neg q U (q \wedge X\Box p)$

- At any execution point, if q is true, p has to be false and vice-versa.

Solution: $\Box(q \Leftrightarrow \neg p)$